

CPDNet: Neural Online Error Monitoring for AI-Aided Kalman Filters

Wenyi Zhang
School of Automation
Northwestern Polytechnical
University
Xi'an, China
zhangwenyi@mail.nwpu.edu.cn

Xiaoyong Ni
Neuro-X Institute
EPFL
Geneva, Switzerland

Nir Shlezinger
School of ECE
Ben-Gurion University
of the Negev
Be'er-Sheva, Israel

Zengfu Wang*
School of Automation
Northwestern Polytechnical
University
Xi'an, China
wangzengfu@nwpu.edu.cn

Abstract—State estimation in dynamical systems is commonly addressed using Kalman filters (KFs) and their variants, which require full characterization of the state-space dynamics. To cope with partially known or complex models, recent works have proposed AI-aided KFs such as KalmanNet, which embed deep neural networks into KF recursions and learn from data to operate under incomplete knowledge. While these architectures exhibit some generalization to unseen dynamics, their performance may degrade under pronounced non-stationarities, where adaptation requires costly online training. In this work, we propose *CPDNet*, a neural module for task-oriented change point detection that monitors AI-aided KFs by leveraging the interpretable innovation process of KalmanNet as indicative features, without relying on ground-truth state information. Numerical experiments on diverse dynamical regimes demonstrate that CPDNet yields accurate degradation indications and facilitates timely adaptation of AI-aided KFs.

Index Terms—AI, Kalman filter, change point detection.

I. INTRODUCTION

State estimation in dynamical systems is a common task shared by various applications in signal processing and control [1]. Standard approaches are based on the Kalman filter (KF) and its variants, which form a widely used family of algorithms thanks to their simple recursive structure, and optimality in linear Gaussian state space (SS) models [2]. These algorithms are inherently model-driven: when the SS parameters change but remain known or reliably identified, the modified parameters can be directly substituted into the filter to adapt to the new regime [3]. However, the reliance of KF-type algorithms on complete specification of the SS model limits their applicability in many practical scenarios, where the dynamics are complex or partially characterized.

The growing success of deep learning led to the emergence of artificial intelligence (AI)-aided state estimators that no longer rely on statistical characterization [4]. In particular, when partial domain knowledge of the system is available, one can exploit it within a model-based deep learning framework [5], where existing approaches combine AI tools for system identification with subsequent model-based tracking [6]; use AI systems in parallel with a KF to provide correction terms [7], [8]; and embed deep neural networks (DNNs) to model the state evolution [9], [10]. A leading family of AI-aided KFs is *KalmanNet* [11] and its variants [12]–[15], which augment KF-type recursions with an inter-

nal DNN for Kalman gain computation, thus learning from data how to implement the filter as a discriminative AI system [16].

While AI-aided filters have demonstrated a degree of generalization to unseen variations, their performance inevitably degrades under sufficient non-stationarities. In classical filters, model adaptation amounts to re-estimating and substituting new parameters, with various tools developed for identifying changes in the SS model via change point detection (CPD) [17], [18], and for subsequent identification of the SS model parameters [19]. However, for AI-aided filters, adaptation requires training rather than estimating the SS model. Although hypernetwork-based approaches can provide limited flexibility [20], and meta-learning [21] with unsupervised training [22] can make re-training more feasible, online adaptation remains a complex task. Therefore, such adaptation should only be triggered when truly necessary, rather than at every variation in the dynamics.

In this work, we introduce a DNN-based mechanism for online monitoring of AI-aided KFs, treating performance degradation (i.e., a substantial rise in state estimation error) as task-oriented CPD. By leveraging the KF's internal innovation process, we eliminate the need for online ground-truth error evaluation. Our proposed *CPDNet* translates these innovations into soft reliability estimates to trigger timely adaptation. We develop a two-stage self-supervised approach that avoids online ground-truth and learns task-oriented boundaries, unlike conventional distribution-shift tests. Numerical results demonstrate CPDNet's robustness across diverse dynamical regimes, while outperforming traditional model-based CPD tests such as BOCD [23] and CuSum [24].

The rest of this paper is organized as follows. Section II presents the system model. CPDNet is presented in Section III and evaluated in Section IV. Section V provides concluding remarks.

II. SYSTEM MODEL

A. State-Space Model

We consider a general discrete-time dynamical system described by a time-varying nonlinear SS model of the form

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}) + \mathbf{w}_t, \quad \mathbf{w}_t \sim P_t(\mathbf{w}), \quad \mathbf{x}_t \in \mathbb{R}^m, \quad (1a)$$

$$\mathbf{y}_t = h(\mathbf{x}_t) + \mathbf{v}_t, \quad \mathbf{v}_t \sim P_t(\mathbf{v}), \quad \mathbf{y}_t \in \mathbb{R}^n. \quad (1b)$$

In (1), $\mathbf{x}_t$ denotes the latent state vector at time t , and $\mathbf{y}_t$ is the corresponding observed measurement. The functions $f(\cdot)$ and $h(\cdot)$ describe the (possibly nonlinear) state transition dynamics and the observation model, respectively. The process noise $\mathbf{w}_t$ and the observation noise $\mathbf{v}_t$ are drawn from temporally independent distributions, denoted by $P_t(\mathbf{v})$ and $P_t(\mathbf{w})$, respectively, that may vary over time to capture non-stationarities.

B. Model-Based and AI-Aided KFs

Extended KF: When the SS model is fully known and the noises are Gaussian, tracking (i.e., estimating $\mathbf{x}_t$ from $\{\mathbf{y}_\tau\}_{\tau \leq t}$) can be reliably performed using model-based filtering variants of the KF for nonlinear dynamics, such as the extended KF (EKF). At each time t , EKF first predicts the state and observations via

$$\hat{\mathbf{x}}_{t|t-1} = f(\hat{\mathbf{x}}_{t-1}), \quad \hat{\mathbf{y}}_{t|t-1} = h(\hat{\mathbf{x}}_{t|t-1}). \quad (2)$$

This is followed by the computation of the Kalman gain matrix $\mathbf{K}_t$ based on linearizations of the SS model and propagation of the second-order moments, leading to the state estimate

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t \cdot \Delta \mathbf{y}_t, \quad \Delta \mathbf{y}_t := \mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}. \quad (3)$$

KalmanNet proposed in [11] augments EKF with a dedicated DNN (typically a recurrent neural network (RNN) [11]) to bypass the need to linearize the SS model and to track second-order moments. This is achieved by replacing the computation of $\mathbf{K}_t$ with a DNN, while training the overall augmented EKF to minimize the empirical mean squared error (MSE).

C. Problem Formulation

KalmanNet enables tracking in settings where the SS model mappings $f(\cdot)$ and $h(\cdot)$ are coarse approximations of the true underlying dynamics, and when the noise distributions $P_t(\mathbf{w})$ and $P_t(\mathbf{v})$ are unknown and non-Gaussian. However, under variations in the SS model not observed in training, e.g., when $P_t(\mathbf{w})$ and $P_t(\mathbf{v})$ notably change in time, its performance may degrade.

Our goal is to design a mechanism which allows identifying when performance degrades without having access to the true state. For design purposes, we assume access to a labeled *offline* dataset $\mathcal{D}$ consisting of input-output pairs, i.e.,

$$\mathcal{D} = \{(\{\mathbf{y}_\tau^{(i)}, \mathbf{x}_\tau^{(i)}\}_{\tau=1}^T)\}_{i=1}^{|\mathcal{D}|}. \quad (4)$$

The offline data is assumed to be decomposable into two subsets: (i) A *stationary* set $\mathcal{D}_s$, originating from a single system regime, in which the noise signals follow (unknown) stationary distributions denoted by $P_0(\mathbf{w})$ and $P_0(\mathbf{v})$; and (ii) a *nonstationary* set $\mathcal{D}_v$, where in each input-output trajectory the distribution of the noises can change in an unknown manner. During inference, the system may encounter different dynamics, but no labeled data or knowledge of the change point is available.

III. CPDNET

The considered problem can be viewed as integrating CPD into AI-aided KFs. While CPD traditionally refers to identifying abrupt changes in the statistics of a sequence [25], applying conventional CPD methods to our setting fails to exploit a key property of KalmanNet – its ability to generalize across

moderate variations in the dynamics [11]. As a result, standard approaches which rely on detecting shifts in the signals, would likely trigger unnecessary adaptation. An alternative perspective is the AI paradigm of concept drift [26]. These tools assess whether a learned model remains suitable over time, with methods such as the DDM [27] and Hotelling's test [28]. However, these approaches are typically designed for evaluation rather than real-time decision-making, and often assume access to labels or performance feedback.

Nevertheless, a unique opportunity for unlabeled CPD arises from the structure of KalmanNet. KalmanNet inherits its formulation from the EKF, whose internal features (e.g., the innovation process) implicitly encode information related to the confidence of the filter and its alignment with the true dynamics. Consequently, by monitoring such internal features and outputs, one can potentially detect when KalmanNet begins to diverge from reliable operation. However, unlike the model-based setting, where the relationship between these features and model validity is tractable and analytically understood, the mapping in KalmanNet is intricate and entangled with the learned network weights. Thus, our CPDNet is based on a DNN (Subsection III-A) and an offline training algorithm (Subsection III-B) designed to capture the relationships between observed behavior and the need for adaptation. While CPDNet is instantiated for KalmanNet-type architectures, its underlying principle—monitoring internal filter features to detect performance degradation—generalizes to any recursive AI-aided filter that exposes an interpretable discrepancy signal.

A. CPD Algorithm

CPDNet is a neural module with parameters ϕ that receives at each time instance t a sequence of processed filtering features, denoted $\mathbf{z}_t$, and produces a soft prediction $\hat{p}_\phi(\mathbf{z}_t) \in [0, 1]$ representing the estimated probability that the filter is no longer aligned with the current dynamics. A high value serves as a data-driven trigger for initiating adaptation in the filtering model.

Input Features: Based on the above rationale, we use the innovation process $\Delta \mathbf{y}_t$ as indicative features for detecting variations. To facilitate learning, we further compress these multivariate quantities into scalar features in the range $[0, 1]$. This is achieved through handcrafted extraction of the form

$$\mathbf{z}_t = \tanh(\gamma \cdot (\sigma(\|\Delta \mathbf{y}_t\|) - 0.5)), \quad (5)$$

where $\sigma(\cdot)$ is the sigmoid function and γ is a hyperparameter. The construction in (5) enables a sequential non-linear mapping, that is, the initial $\sigma(\|\Delta \mathbf{y}_t\|) - 0.5$ transforms the ℓ_2 norm into a normalized "influence" score within $[0, 0.5]$, while establishing a zero influence for zero input. Subsequently, the $\tanh(\cdot)$ function performs a second-stage non-linear mapping to a probabilistic range. To reduce sensitivity to isolated anomalies, we aggregate the transformed features over a sliding window of length $\delta \geq 1$, such that the features processed by the CPDNet are given by

$$\mathbf{z}_t = [\mathbf{z}_{t-\delta+1}, \dots, \mathbf{z}_t]. \quad (6)$$

Architecture: CPDNet is structured to learn to exploit temporal dependencies between the entries of $\mathbf{z}_t$, with an output layer designed to be geared towards not triggering

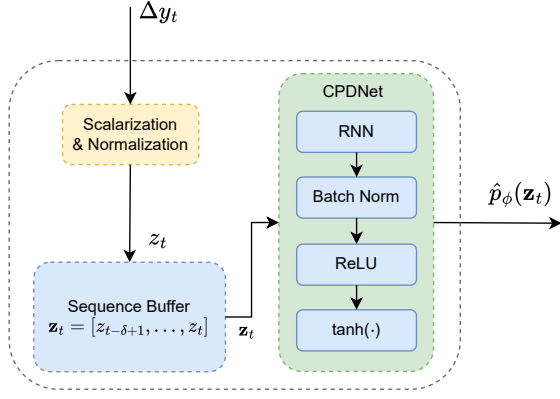


Fig. 1: CPDNet block diagram

adaptation when one is not required. As such, it consists of an interconnection of an RNN, applied to the windowed time sequence z_t in a many-to-one fashion, and a fully-connected layer that produces scalar outputs, with an output ReLU activation followed by a scaled tanh to ensure probability outputs in $[0, 1]$, while encouraging 0 outputs (by applying ReLU before tanh) to avoid unnecessary adaptation. The resulting architecture is illustrated in Fig. 1.

B. Training

CPDNet is designed to monitor KalmanNet without online labels. Yet, CPDNet itself is trained in advance using the data in (4) and by formulating a supervised loss measure. Training is done in two stages: (i) Stationary reference pre-training; and (ii) Nonstationary change-detect training. We next elaborate on these stages.

Stationary Reference Pre-Training: The first stage trains a reference KalmanNet model using the stationary $\mathcal{D}_s$. The purpose of introducing this auxiliary model is to facilitate both the generation of training data and the construction of a supervised-style loss function, which is used to train CPDNet.

Letting θ denote the parameters of the reference KalmanNet, these parameters are trained guided by the MSE loss on $\mathcal{D}_s$,

$$\mathcal{L}_{\mathcal{D}_s}^{\text{MSE}}(\theta) = \frac{1}{|\mathcal{D}_s|T} \sum_{i=1}^{|\mathcal{D}_s|} \sum_{t=1}^T \left\| \mathbf{x}_t^{(i)} - \hat{\mathbf{x}}_t(\mathbf{y}_t^{(i)}; \theta) \right\|^2. \quad (7)$$

Nonstationary Change-Detect Training: Next, we fix θ , and train CPDNet to monitor its behavior when applied to the nonstationary data $\mathcal{D}_v$. Since $\mathcal{D}_v$ contains both observations and state trajectories, we can assess the accuracy of the filter, and evaluate CPDNet in training by its ability to identify degradation.

To convert the error of KalmanNet, i.e., $\left\| \mathbf{x}_t^{(i)} - \hat{\mathbf{x}}_t(\mathbf{y}_t^{(i)}; \theta) \right\|$, into a probabilistic quantity, we repeat the mapping in (5), setting

$$p_t^{(i)}(\theta) = \tanh \left(\gamma \left(\sigma \left(\left\| \mathbf{x}_t^{(i)} - \hat{\mathbf{x}}_t(\mathbf{y}_t^{(i)}; \theta) \right\| \right) - 0.5 \right) \right). \quad (8)$$

The probability (8) approaches one when the ℓ_2 error of KalmanNet is large, and is thus used as labels for CPDNet. Since $\mathcal{D}_v$ contains nonstationary data, applying KalmanNet trained on the stationary $\mathcal{D}_s$ is expected to lead to fluctuations in its

performance, which CPDNet should learn to detect. To formulate the resulting loss, we write the input features to CPDNet based on the innovation process produced by KalmanNet with parameters θ applied to measurements $\mathbf{y}_t^{(i)}$ as $z_t^{(i)}(\theta)$. We can now treat CPDNet as a binary detection network, and train it based on the binary cross-entropy (BCE) loss evaluated over $\mathcal{D}_v$, namely

$$\mathcal{L}_{\mathcal{D}_v}^{\text{BCE}}(\phi; \theta) = \frac{-1}{|\mathcal{D}_v|(T-\delta+1)} \sum_{i=1}^{|\mathcal{D}_v|} \sum_{t=\delta}^T p_t^{(i)}(\theta) \log \hat{p}_\phi(z_t^{(i)}(\theta)) + (1-p_t^{(i)}(\theta)) \log(1-\hat{p}_\phi(z_t^{(i)}(\theta))). \quad (9)$$

The overall training procedure using stochastic gradient descent (SGD)-based learning is summarized as Algorithm 1.

Algorithm 1: CPDNet Training using SGD

Init : Dataset $\mathcal{D} = \mathcal{D}_s \cup \mathcal{D}_v$; Hyperparameters δ, γ
Step-size $\tilde{\eta}$, batches L , epochs $e_{\max}$; Initial ϕ
// Stationary Reference Pre-Training
1 Initialize reference KalmanNet θ ;
2 **for** epoch = 0, 1, ..., $e_{\max}$ **do**
3 Randomly divide $\mathcal{D}_s$ into L batches $\{\mathcal{D}_s^l\}_{l=1}^L$;
4 **for** $l = 1, \dots, L$ **do**
5 Compute KalmanNet MSE loss (7) on $\mathcal{D}_s^l$;
6 Update $\theta \leftarrow \theta - \tilde{\eta} \nabla_{\theta} \mathcal{L}_{\mathcal{D}_s^l}^{\text{MSE}}(\theta)$;
// Nonstationary Training
7 **for** epoch = 0, 1, ..., $e_{\max}$ **do**
8 Randomly divide $\mathcal{D}_v$ into L batches $\{\mathcal{D}_v^l\}_{l=1}^L$;
9 **for** $l = 1, \dots, L$ **do**
10 Compute CPDNet BCE loss (9) on $\mathcal{D}_v^l$;
11 Update $\phi \leftarrow \phi - \tilde{\eta} \nabla_{\phi} \mathcal{L}_{\mathcal{D}_v^l}^{\text{BCE}}(\phi; \theta)$;
Output: Trained CPDNet ϕ .

IV. EXPERIMENTS

A. Experimental Setup

We evaluate CPDNet under two SS models: (i) a linear Gaussian model with $f(\mathbf{x}) = \mathbf{F}\mathbf{x}$ and $h(\mathbf{x}) = \mathbf{H}\mathbf{x}$, where $\mathbf{w}_t$ and $\mathbf{v}_t$ are zero-mean Gaussian noises with covariances $\mathbf{Q}_1$ and $\mathbf{R}_1$, respectively. We adopt a constant-acceleration model [1] with a sampling interval of 0.01 and $\mathbf{R} = 1$; (ii) the nonlinear Lorenz attractor model [11] with the same noise assumptions. KalmanNet is pretrained using 1000 trajectories from the initial SS model, and CPDNet is trained with $|\mathcal{D}_v| = 100$ nonstationary and $|\mathcal{D}_s| = 100$ stationary trajectories drawn from the same distribution.

We simulate three forms of temporal shifts: (i) *Abrupt changes*, where we insert a random change point $t_{\text{change}} \sim \mathcal{U}[1, T]$, and scale the varying parameter by λ . For example, abrupt changes in $\mathbf{Q}$ are set as $\mathbf{Q}_1$ if $t \leq t_{\text{change}}$ and $\lambda \cdot \mathbf{Q}_1$ otherwise. (ii) *Gradual changes*, where the varying parameter scales on each time instance. For example, gradual changes in $\mathbf{Q}$ are set as $\mathbf{Q}_1$ if $t \leq t_{\text{change}}$ and $\lambda^{t-t_{\text{change}}} \cdot \mathbf{Q}_1$ otherwise. (iii) *Rotated changes*, where we induce a 2D rotation by 1° to the state

TABLE I: MSE in predicting KalmanNet error for the linear model case.

Parameter	Change Type (λ)	MSE (dB)
Q	Abrupt (100/300/500)	-26.26/-22.20/-18.18
	Gradual	-20.66
R	Abrupt (1.2/1.3/1.5)	-23.75/-23.06/-20.67
	Gradual	-19.71
F	Abrupt (1.01/0.95)	-23.33/-17.86
H	Abrupt (1.01/0.95)	-25.73/-17.86

TABLE II: MSE in predicting KalmanNet error for the nonlinear model case.

Parameter	Change Type(λ)	MSE (dB)
Q	Abrupt (1000/500/300)	4.97/3.69/3.19
	Gradual	6.09
R	Abrupt (50/100/150)	4.00/4.78/4.48
	Gradual	1.02
$f(\cdot)$	Rotated	6.41
H	Rotated	-3.78

evolution or the observation model. We set $\gamma = 19$ and window size equal to 5, longer window size yields more stable results but increases latency, and vice versa.

We evaluate CPDNet by comparing its predictions with the error signals obtained from KalmanNet. Note that these errors are not actual probabilities, but rather probabilistic representations of the state estimation error produced by KalmanNet.

Table I reports the MSE between CPDNet predictions and the probabilistic KalmanNet error for the linear case, demonstrating high precision across all scenarios (e.g., -26.26 dB for abrupt **Q** changes). Fig. 2 illustrates how CPDNet outputs reliably track the onset of degradation.

Similar trends are reported for the nonlinear case in Table II and Fig. 3. In Table II, the task is significantly more challenging, reflected by higher MSE values. Nonetheless, CPDNet still effectively detects various changes. Notably, it demonstrates a strong ability to identify gradual variations in **R** and complex rotational transformations in the observation matrix (**H**), achieving an MSE of 1.02 dB and -3.78 dB, respectively. Meanwhile, the results show that the CPDNet rarely yields false alarms in the absence of change points.

We conclude our study by contrasting CPDNet with classical CPD methods. In view of the limitations of conventional approaches [29], [30], we only focus on the linear SS model in (i). and compare CPDNet with BOCD and the CuSum under the linear model described in (i). A change point is declared when the CPDNet output probability first exceeds a threshold of 0.15. The results are reported in Table III, where MSE quantifies the

TABLE III: Comparison of the MSE performance of CPDNet, BOCD, and CuSum. Lower values indicate better performance.

Parameter (λ)	CPDNet	BOCD	CuSum
Q (250 / 300 / 350)	18.4 / 11.8 / 18.2	20.5 / 17.2 / 20.0	20.50 / 13.53 / 31.22
R (4 / 3 / 2)	6.6 / 8.1 / 4.2	24.9 / 13.1 / 19.5	12.33 / 17.05 / 5.81
F (1.03 / 1.04 / 1.05)	11.2 / 4.4 / 8.3	19.4 / 15.5 / 27.1	18.86 / 29.99 / 19.64
H (1.1 / 1.15 / 1.2)	6.7 / 13.7 / 10.5	22.6 / 26.2 / 19.6	13.34 / 36.89 / 31.56

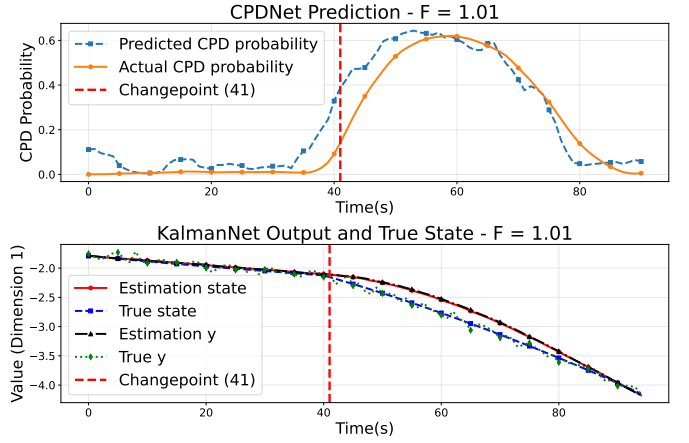


Fig. 2: Abrupt change of the state transition matrix **F** by a factor of $\lambda = 1.01$. The top panel shows the predicted and actual change point probabilities, together with the ground-truth change point. The bottom panel compares the KalmanNet estimation and the true state/output for the first component of the state vector.

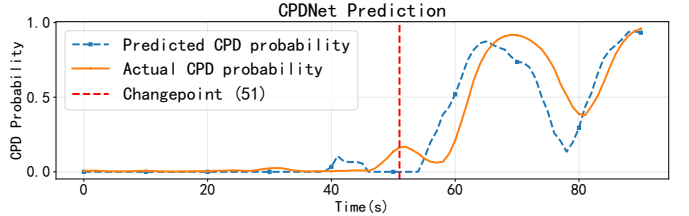


Fig. 3: Prediction accuracy of CPDNet under a nonlinear scenario with rotational mismatch.

temporal deviation between predicted and actual change points. Table III shows that CPDNet consistently achieves the lowest MSE across all parameter variations. CPDNet reduces estimation errors by up to 85.3% compared to CuSum (e.g., at **F** with $\lambda = 1.04$) and maintains substantial margins over BOCD, such as a 78.5% reduction at **R** ($\lambda = 2$). While the baselines exhibit significant performance degradation under high-dynamic shifts (e.g., **H** at $\lambda = 1.2$), CPDNet demonstrates remarkable stability.

V. CONCLUSION

We presented CPDNet, a neural module for online error monitoring of AI-aided Kalman filters. By leveraging the innovation process of KalmanNet, CPDNet enables task-oriented CPD without requiring ground-truth state information. Numerical experiments on both linear and nonlinear dynamical systems demonstrated its ability to provide accurate degradation indications. Future work involves extending CPDNet to non-recursive estimators and unsupervised online settings without explicit innovations or offline labels.

VI. ACKNOWLEDGMENT

This work was supported in part by the National Natural Science Foundation of China (Grants No. 62473317, U21B2008).

REFERENCES

- [1] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation With Applications to Tracking and Navigation: Theory Algorithms and Software*. John Wiley & Sons, 2004.
- [2] J. Durbin and S. J. Koopman, *Time series analysis by state space methods*. Oxford University Press, 2012.
- [3] S. Gannot and A. Yeredor, “The Kalman filter,” *Springer Handbook of Speech Processing*, pp. 135–160, 2008.
- [4] N. Shlezinger *et al.*, “Artificial intelligence-aided Kalman filters: AI-augmented designs for Kalman-type algorithms,” *IEEE Signal Process. Mag.*, vol. 42, no. 3, pp. 52–76, 2025.
- [5] N. Shlezinger, J. Whang, Y. C. Eldar, and A. G. Dimakis, “Model-based deep learning,” *Proc. IEEE*, vol. 111, no. 5, pp. 465–499, 2023.
- [6] T. Imbiriba, O. Straka, J. Duník, and P. Closas, “Augmented physics-based machine learning for navigation and tracking,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. 60, no. 3, pp. 2692–2704, 2024.
- [7] V. G. Satorras, Z. Akata, and M. Welling, “Combining generative and discriminative models for hybrid inference,” in *Adv. Neural Inf. Process. Syst.*, 2019, pp. 13 802–13 812.
- [8] K. Pratik *et al.*, “Neural augmentation of Kalman filter with hypernetwork for channel tracking,” in *Proc. IEEE Glob. Commun. Conf.*, 2021.
- [9] A. Ghosh, A. Honoré, and S. Chatterjee, “DANSE: Data-driven non-linear state estimation of model-free process in unsupervised learning setup,” *IEEE Trans. Signal Process.*, vol. 72, pp. 1824–1838, 2024.
- [10] A. Ghosh, M. Abdalmoaty, S. Chatterjee, and H. Hjalmarsson, “DeepBayes—an estimator for parameter estimation in stochastic nonlinear dynamical models,” *Automatica*, vol. 159, p. 111327, 2024.
- [11] G. Revach *et al.*, “KalmanNet: Neural network aided Kalman filtering for partially known dynamics,” *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.
- [12] G. Revach, X. Ni, N. Shlezinger, R. J. van Sloun, and Y. C. Eldar, “RTSNet: Learning to smooth in partially known state-space models,” *IEEE Trans. Signal Process.*, vol. 71, pp. 4441–4456, 2023.
- [13] I. Buchnik *et al.*, “Latent-KalmanNet: Learned Kalman filtering for tracking from high-dimensional signals,” *IEEE Trans. Signal Process.*, vol. 72, pp. 352–367, 2023.
- [14] J. Wang, X. Geng, and J. Xu, “Nonlinear Kalman filtering based on self-attention mechanism and lattice trajectory piecewise linear approximation,” *arXiv preprint arXiv:2404.03915*, 2024.
- [15] I. Buchnik *et al.*, “GSP-KalmanNet: Tracking graph signals via neural-aided Kalman filtering,” *IEEE Trans. Signal Process.*, vol. 72, pp. 3700–3716, 2024.
- [16] N. Shlezinger and T. Routtenberg, “Discriminative and generative learning for linear estimation of random signals,” *IEEE Signal Process. Mag.*, vol. 40, no. 6, pp. 75–82, 2023.
- [17] K. Tsampourakis and V. Elvira, “Approximating the likelihood ratio in linear-Gaussian state-space models for change detection,” in *Proc. IEEE ICASSP*, 2022.
- [18] R. Alami *et al.*, “Restarted Bayesian online change-point detector achieves optimal detection delay,” in *Proc. Int. Conf. Mach. Learn.*, 2020.
- [19] R. Mehra, “Approaches to adaptive filtering,” *IEEE Trans. Autom. Control*, vol. 17, no. 5, pp. 693–698, 1972.
- [20] X. Ni, G. Revach, and N. Shlezinger, “Adaptive KalmanNet: Data-driven Kalman filter with fast adaptation,” in *Proc. IEEE ICASSP*, 2024.
- [21] S. Chen *et al.*, “MAML-KalmanNet: A neural network-assisted Kalman filter based on model-agnostic meta-learning,” *IEEE Trans. Signal Process.*, vol. 73, pp. 988–1003, 2025.
- [22] G. Revach *et al.*, “Unsupervised learned Kalman filtering,” in *Proc. IEEE EUSIPCO*, 2022.
- [23] R. P. Adams and D. J. C. MacKay, “Bayesian online changepoint detection,” *arXiv preprint arXiv:0710.3742*, 2007.
- [24] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
- [25] S. Aminikhahhahi and D. J. Cook, “A survey of methods for time series change point detection,” *Knowl. Inf. Syst.*, vol. 51, no. 2, pp. 339–367, 2017.
- [26] J. Lu *et al.*, “Learning under concept drift: A review,” *IEEE Trans. Knowl. Data Eng.*, vol. 31, pp. 2346–2363, 2018.
- [27] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, “Learning with drift detection,” in *Proc. Braz. Symp. Artif. Intell.* Springer, 2004, pp. 286–295.
- [28] N. Uzlaner, T. Raviv, N. Shlezinger, and K. Todros, “Concept drift detection for deep learning aided receivers in dynamic channels,” in *Proc. IEEE SPAWC*, 2024.
- [29] Z. Ebrahimzadeh, M. Zheng, S. Karakas, and S. Kleinberg, “Deep learning for multi-scale changepoint detection in multivariate time series,” *arXiv preprint arXiv:1905.06913*, 2019.
- [30] F. Aouissaoui and J. Ngatchou-Wandji, “Weak change detection: A review,” *Mathematics*, vol. 14, no. 8, 2026.